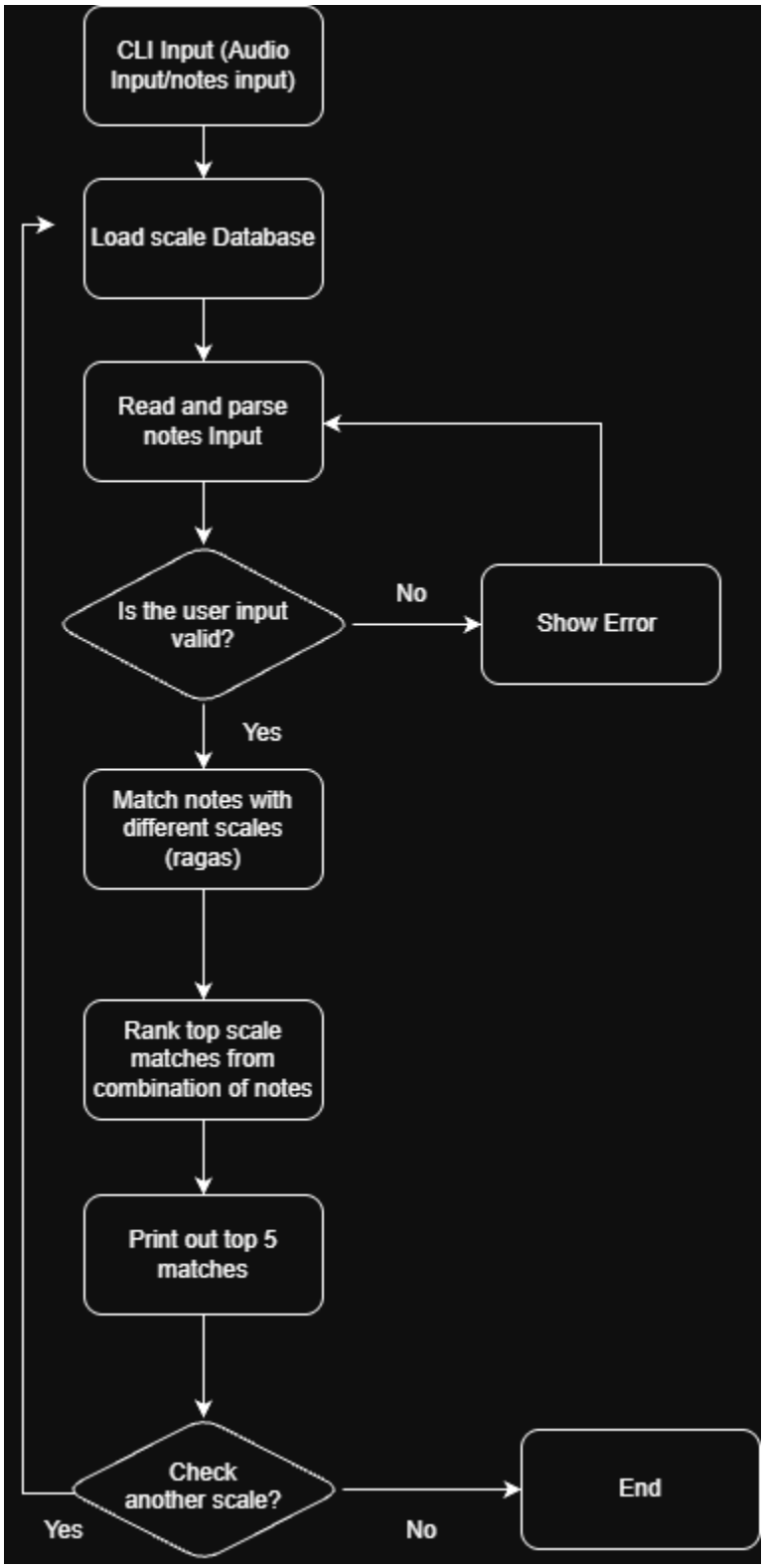
	Urgent	Not Urgent
Important	**Do (your MVP)** - Parse typed swara input (S R2 G3 M1 P D2 N3 S) - Raga database: the 72 melakartas, generated by rule or loaded from a CSV - Scoring: percentage of input notes that fit each raga - Print the top 3 to 5 matches, ranked	**Schedule (V2.0+)** - Check arohanam/avarohanam order, not just the note set - Add janya ragas to the database - “Guess the raga” quiz mode - Read note sequences from a text file - Show a raga’s notes when you look it up by name
Not Important	**Delegate (use a library)** - CSV parsing (OpenCSV or buffered reader) - Argument handling - Table-style output formatting	**Delete (cut it)** - Recognizing ragas from raw audio - Gamaka (ornamentation) detection - GUI or web interface - Playing the notes back as sound

Tech Stack

- Language
  - Java, built with Maven
- Libraries, packages, and external tools
  - TarsosDSP - pitch detection
  - javax.sound.sampled
  - Gson - reading gson
  - JUnit
  - ffmpeg (command-line tool to convert MP3/M4A recordings to WAV)
- Data storage
  - Ragas.json file storing each raga’s name, arohanam, avarohanam, and swara set
  - WAV audio files stored in samples folder
  - Results printed to the console, with an optional CSV output file
  - No database needed
- Data sources
  - Raga definitions entered by hand from public references such as the Melakarta chart
  - My own audio recordings as test clips, and optionally the Saraga dataset for additional Carnatic audio
- Other technical details
  - Git/GitHub for version control
  - Runs as a command-line program: the user gives an audio file and tonic (Sa), and the program prints the most likely raga matches

Flowchart


CLI Input (Audio Input/notes input)
Load scale Database
Read and parse notes Input
Is the user input valid?
No
Show Error
Yes
Match notes with different scales (ragas)
Rank top scale matches from combination of notes
Print out top 5 matches
Check another scale?
Yes
No
End